

```

        m >>= 1;
    }
    j += m;
}

```

Here begins the Danielson-Lanczos section of the routine.

```

mmax=2;
while (nn > mmax) {
    istep=mmax << 1;
    theta=isign*(6.28318530717959/mmax);
    wtemp=sin(0.5*theta);
    wpr = -2.0*wtemp*wtemp;
    wpi=sin(theta);
    wr=1.0;
    wi=0.0;
    for (m=1;m<mmax;m+=2) {
        for (i=m;i<=nn;i+=istep) {
            j=i+mmax;
            tempr=wr*data[j-1]-wi*data[j];
            tempi=wr*data[j]+wi*data[j-1];
            data[j-1]=data[i-1]-tempr;
            data[j]=data[i]-tempi;
            data[i-1] += tempr;
            data[i] += tempi;
        }
        wr=(wtemp*wr)*wpr-wi*wpi+wr;
        wi=wi*wpr+wtemp*wpi+wi;
    }
    mmax=istep;
}

```

Outer loop executed $\log_2 n$ times.

Initialize the trigonometric recurrence.

Here are the two nested inner loops.

This is the Danielson-Lanczos formula:

Trigonometric recurrence.

For an interface at a slightly higher level, we can overload the bare `four1` with equivalent functions that input and output data as either a `VecDoub` of length $2N$ or a `VecComplex` of length N :

```

void four1(VecDoub_IO &data, const Int isign) {
    Overloaded interface to four1. Replaces the vector data, a complex vector of length  $N$  stored
    as a real vector of twice that length, by its discrete Fourier transform, with components in
    wraparound order, if isign is 1; or by  $N$  times the inverse Fourier transform, if isign is -1.
    four1(&data[0],data.size()/2,isign);
}

void four1(VecComplex_IO &data, const Int isign) {
    Overloaded interface to four1. Replaces the vector data, a complex vector of length  $N$  stored
    as such, by its discrete Fourier transform, with components in wraparound order, if isign is 1;
    or by  $N$  times the inverse Fourier transform, if isign is -1.
    four1((Doub*)&data[0],data.size(),isign);
}

```

fourier.h

In these overloaded versions, however, you are still responsible for decoding on your own the wraparound order. To get an interface that takes care of that for you, we can define an object `WrapVecDoub` that creates a real vector (or binds a reference to an existing one) and then defines methods for addressing the vector as if it were a complex vector of half the size. Since the `WrapVecDoub` object also knows about wraparound periodicity, you can access frequencies either by subscripts $[0 \dots n-1]$ or $[-n/2 \dots n/2-1]$, or, for that matter, any n consecutive complex components. The object has a conversion operator to `VecDoub`, so you can (e.g.) send it directly to `four1`.

fourier.h

```
struct WrapVecDoub {
```

Object for accessing a VecDoub as if it were a complex vector of half the length, with wraparound periodicity.

```
    VecDoub vvec;                Used when data are stored internally.
    VecDoub &v;
    Int n, mask;
```

```
    WrapVecDoub(const Int nn) : vvec(nn), v(vvec), n(nn/2),
    mask(n-1) {validate();}
```

Constructor. Declare a new vector with *nn* real components (half as many complex).

```
    WrapVecDoub(VecDoub &vec) : v(vec), n(vec.size()/2),
    mask(n-1) {validate();}
```

Constructor. Bind the data in an existing vector *vec* for access as if complex.

```
    void validate() {if (n&(n-1)) throw("vec size must be power of 2");}
```

```
    inline Complex& operator[] (Int i) {return (Complex &)v[(i&mask) << 1];}
```

Reduce any integer *i* to the periodic range $[0..n]$ and return that complex component. Can also be an l-value.

```
    inline Doub& real(Int i) {return v[(i&mask) << 1];}
```

As above, but return the real part only. Can also be an l-value.

```
    inline Doub& imag(Int i) {return v[(i&mask) << 1]+1];}
```

As above, but return the imaginary part only. Can also be an l-value.

```
    operator VecDoub&() {return v;}
```

Conversion operator. Allows a WrapVecDoub object to be sent to any function that expects a VecDoub.

```
};
```

Here are some sample lines of code (not a useful program) that show how WrapVecDoub can be used:

```
Int j,n=256;
VecDoub dat(2*n);
WrapVecDoub data1(dat), data2(2*n);
for (j=0;j<n;j++) {
    data1[j] = Complex(... , ...);
    data2.real(j) = ... ;
    data2.imag(j) = ... ;
}
four1(data1,1);
four1(data2,1);
for (j=-n/2;j<n/2;j++) {
    ... = data1.real(j);
    ... = data2[j];
}
```

256 complex components, e.g.
A real vector to hold them.
Examples of the two constructors.
Loop over complex components.
Set a complex value directly,
or set real and imag separately.

Invokes four1(VecDoub&,Int) through the
conversion operator.
Can address negative frequencies directly!
Get real part of component *j*.
Get component as a complex value.

12.2.2 Decomposing the FFT for Parallelism

It is possible to decompose the calculation of an FFT of size N into a set of smaller FFTs that can be done independently of one another. This can be useful either to achieve parallelism or to allow more versatile memory management. The basic idea is to address the input array as if it were a two-dimensional array of size $m \times M$, where $N = mM$ and N , m , and M are all powers of 2. Then the components of f can be addressed as

$$f[Jm + j], \quad 0 \leq j < m, \quad 0 \leq J < M \quad (12.2.5)$$

where the j index changes more rapidly, the J index more slowly, and brackets denote C++ subscripts.

What we want to compute is the FFT of the original array of length N , which we can write as

$$F[kM + K] \equiv \sum_{j,J} e^{2\pi i(kM+K)(Jm+j)/(Mm)} f(Jm + j), \quad (12.2.6)$$

$$0 \leq k < m, \quad 0 \leq K < M$$

You can see that the indices k and K address the desired result (FFT of the original array), with K varying more rapidly.

From equation (12.2.6) it is easy to verify the identity

$$F[kM + K] = \sum_j \left\{ e^{2\pi i j k / m} \left[e^{2\pi i j K / (Mm)} \left(\sum_J e^{2\pi i J K / M} f(Jm + j) \right) \right] \right\} \quad (12.2.7)$$

But this, reading it from the innermost operation outward, is just the method that we need:

- For each value of j form a vector of input values whose components vary with J , $0 \leq J < M$. This is basically a transpose operation.
- FFT each such vector. (OK to do in parallel.) Notationally, the J index now becomes a K index.
- Multiply each component by a phase factor $\exp[2\pi i j K / (Mm)]$.
- Rearrange the data so that they are accessible as a set of vectors whose components vary with j , $0 \leq j < m$, another transpose operation.
- FFT each such vector. (OK to do in parallel.) The j index now becomes a k index.
- The answer is now available as $F[kM + K]$. It takes a third transpose operation to get it back into the desired order (with k varying most rapidly).

Even though two FFTs are performed on each element, the operations count is about the same as for the ordinary FFT: The first set of FFTs scales as $N \log M$, the second set as $N \log m$, and the total is thus $N \log(Mn) = N \log N$.

For further discussion, see [2], where the above is called the *six-step framework*. You can easily eliminate the first two of the three transpose operations by writing a new `four1` routine with an additional *stride* argument, specifying the constant increment between logically “next” components. The stride will be m for the first set of FFTs, and 1 for the second set. An algorithm very similar to this is called the *four-step framework*. See [2,3] for more details.

Related decompositions, called *zoom transforms*, can be used to get an approximation to the spectrum of a long data stream, at high resolution in only certain frequency bands. See [4-6].

12.2.3 Other FFT Algorithms

We should mention that there are a number of variants on the basic FFT algorithm given above. As we have seen, that algorithm first rearranges the input elements into bit-reverse order, then builds up the output transform in $\log_2 N$ iterations. In the literature, this sequence is called a *decimation-in-time* or *Cooley-Tukey* FFT